

**IMPLEMENTASI DAN ANALISIS PERFORMA ALGORITMA RADIX SORT (LSD
DAN MSD) SERTA TERNARY SEARCH PADA PENGOLAHAN DATA NIM
MAHASISWA**

Dosen Pengampu : Ali Akbar Lubis, S.Kom., M.Ti.



Disusun oleh:

Kelompok 6 :

BORKATI HALAWA	5252451006
NABILA	5251151001
REFANI PURBA	5251151003
SYIFA ALYA ANISAH	5251151015

PROGRAM STUDI

PENDIDIKAN TEKNOLOGI INFORMATIKA DAN KOMPUTER

UNIVERSITAS NEGERI MEDAN

TP 2026

DAFTAR ISI

DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 LATAR BELAKANG	1
1.2 Identifikasi Masalah.....	2
1.3 Rumusan Masalah	2
1.4 Tujuan Penelitian.....	2
BAB II LANDASAN TEORI.....	3
2.1 Radix Sort LSD (Least Significant Digit)	3
2.2 Radix Sort MSD (Most Significant Digit)	3
2.3 Ternary Search	4
BAB III PSEUDOCODE ALGORITMA.....	5
3.1 Pseudocode Radix Sort LSD (Least Significant Digit)	5
3.2 Pseudocode Radix Sort MSD (Most Significant Digit)	6
3.3 Pseudocode Ternary Search.....	7
BAB IV DESAIN DAN IMPLEMENTASI PROGRAM	9
4.1 Arsitektur Program	9
4.2 Struktur Data yang Digunakan	10
4.3.1 Modul Input Data	10
4.3.2 Modul Radix Sort LSD	11
4.3.3 Modul Radix Sort MSD	11
4.3.4 Modul Bubble Sort	11
4.3.5 Modul Ternary Search.....	12
4.3.6 Modul Export CSV	12
4.4 Tampilan Program Saat Dieksekusi	12
BAB V STUDI KASUS	15
5.1 Deskripsi Kasus Nyata	15
5.2 Dataset yang Digunakan	15
5.3 Pembahasan Hasil Pengujian	17
BAB VI ANALISIS HASIL EKSPERIMEN	19

6.1 Hasil Pengujian dan Perbandingan Performa	19
6.2 Grafik Perbandingan Performa Algoritma.....	20
6.3 Analisis Temuan	20
BAB VII KESIMPULAN	22
7.1 Kesimpulan	22
7.2 Saran	23
REFERENSI.....	24

BAB I PENDAHULUAN

1.1 LATAR BELAKANG

Perkembangan teknologi informasi telah meningkatkan jumlah data yang harus dikelola dalam berbagai bidang, termasuk bidang pendidikan. Salah satu data yang penting dalam lingkungan akademik adalah Nomor Induk Mahasiswa (NIM) yang digunakan sebagai identitas unik setiap mahasiswa. Seiring bertambahnya jumlah mahasiswa, pengelolaan data NIM memerlukan metode yang mampu mengolah data secara cepat, tepat, dan efisien agar proses administrasi maupun pencarian informasi dapat dilakukan dengan baik.

Dalam pengolahan data, proses **pengurutan (sorting)** dan **pencarian (searching)** merupakan dua kegiatan yang sangat penting. Pengurutan data bertujuan untuk menyusun data dalam urutan tertentu sehingga lebih mudah dikelola dan dicari. Salah satu algoritma sorting yang sering digunakan adalah **Bubble Sort**, namun algoritma ini kurang efisien untuk data dalam jumlah besar karena membutuhkan banyak perbandingan dan pertukaran elemen. Oleh karena itu, diperlukan algoritma yang lebih efektif untuk menangani data numerik seperti NIM mahasiswa.

Salah satu algoritma yang dapat digunakan adalah **Radix Sort**, yaitu algoritma pengurutan non-perbandingan (*non-comparison sorting*) yang bekerja berdasarkan digit angka. Pada penelitian ini digunakan dua variasi Radix Sort, yaitu **Least Significant Digit (LSD)** dan **Most Significant Digit (MSD)**. Radix Sort LSD melakukan pengurutan mulai dari digit paling kanan menuju digit paling kiri, sedangkan Radix Sort MSD memulai pengurutan dari digit paling kiri dan melanjutkannya secara rekursif. Kedua metode tersebut memiliki karakteristik yang berbeda sehingga perlu dianalisis untuk mengetahui performa terbaik dalam pengolahan data NIM.

Setelah proses pengurutan selesai, pencarian data dilakukan menggunakan **Ternary Search**, yaitu algoritma pencarian yang membagi ruang pencarian menjadi tiga bagian pada setiap iterasi. Penggunaan Ternary Search pada data yang telah terurut dapat meningkatkan kecepatan pencarian dibandingkan metode pencarian sederhana. Oleh karena itu, penelitian ini dilakukan untuk mengimplementasikan serta menganalisis performa **Radix Sort LSD**, **Radix Sort MSD**, dan **Ternary Search** dalam pengolahan data NIM mahasiswa, sehingga dapat diketahui algoritma yang paling efektif dan efisien untuk digunakan.

1.2 Identifikasi Masalah

Berdasarkan latar belakang yang telah diuraikan, dapat diidentifikasi beberapa permasalahan sebagai berikut:

1. Pengolahan data NIM mahasiswa dalam jumlah besar membutuhkan algoritma pengurutan yang efisien.
2. Belum diketahui algoritma Radix Sort yang memiliki performa lebih baik antara metode LSD dan MSD.
3. Diperlukan metode pencarian yang cepat setelah data berhasil diurutkan.
4. Perlu dilakukan analisis dan perbandingan performa antara algoritma Radix Sort dan algoritma sorting konvensional seperti Bubble Sort.

1.3 Rumusan Masalah

1. Bagaimana cara mengimplementasikan algoritma Radix Sort LSD untuk mengurutkan data NIM mahasiswa?
2. Bagaimana cara mengimplementasikan algoritma Radix Sort MSD untuk mengurutkan data NIM mahasiswa?
3. Bagaimana cara mengimplementasikan algoritma Ternary Search untuk mencari data pada kumpulan data yang telah terurut?
4. Bagaimana perbandingan performa antara Radix Sort LSD, Radix Sort MSD, dan Bubble Sort berdasarkan jumlah operasi dan waktu eksekusi?
5. Algoritma manakah yang paling efisien untuk digunakan dalam pengolahan data NIM mahasiswa?

1.4 Tujuan Penelitian

1. Mengimplementasikan algoritma Radix Sort LSD pada data NIM mahasiswa.
2. Mengimplementasikan algoritma Radix Sort MSD pada data NIM mahasiswa.
3. Mengimplementasikan algoritma Ternary Search pada data yang telah diurutkan.
4. Menganalisis performa algoritma Radix Sort LSD, Radix Sort MSD, dan Bubble Sort.
5. Menentukan algoritma yang paling efektif dan efisien dalam pengolahan data NIM mahasiswa.

BAB II LANDASAN TEORI

2.1 Radix Sort LSD (Least Significant Digit)

Radix Sort LSD (*Least Significant Digit*) merupakan algoritma pengurutan non-perbandingan (*non-comparison sorting algorithm*) yang melakukan pengurutan berdasarkan digit dari posisi paling rendah atau digit paling kanan menuju digit yang lebih tinggi. Berbeda dengan algoritma sorting konvensional yang membandingkan nilai antar elemen secara langsung, Radix Sort LSD mengelompokkan data berdasarkan nilai digit pada setiap posisi secara bertahap hingga seluruh digit selesai diproses. Prinsip kerja Radix Sort LSD dimulai dengan mengurutkan data berdasarkan digit satuan, kemudian dilanjutkan ke digit puluhan, ratusan, dan seterusnya hingga digit terbesar. Sebagai contoh, data 170, 45, 75, 90, 802, 24, 2, dan 66 akan diurutkan berdasarkan digit satuan terlebih dahulu, kemudian puluhan, dan akhirnya ratusan hingga menghasilkan urutan yang benar.

Keunggulan utama Radix Sort LSD adalah kemampuannya mengurutkan data numerik dalam jumlah besar dengan efisien tanpa melakukan perbandingan langsung antar elemen. Selain itu, algoritma ini bersifat stabil (*stable sort*), sehingga urutan data yang memiliki nilai sama tetap dipertahankan. Oleh karena itu, Radix Sort LSD banyak digunakan pada data numerik seperti Nomor Induk Mahasiswa (NIM), kode pos, nomor identitas, dan data lain yang memiliki format digit tetap. Meskipun demikian, Radix Sort LSD memiliki beberapa keterbatasan. Algoritma ini membutuhkan memori tambahan untuk proses pengelompokan data ke dalam bucket serta kurang efisien apabila digunakan pada data yang memiliki panjang digit sangat besar. Selain itu, implementasinya relatif lebih kompleks dibandingkan algoritma sorting sederhana seperti Bubble Sort.

2.2 Radix Sort MSD (Most Significant Digit)

Radix Sort MSD (*Most Significant Digit*) merupakan algoritma pengurutan non-perbandingan yang melakukan proses pengurutan dimulai dari digit paling signifikan atau digit paling kiri terlebih dahulu. Setelah data dikelompokkan berdasarkan digit pertama, proses pengurutan dilanjutkan secara rekursif pada digit berikutnya hingga seluruh data berada dalam urutan yang benar. Berbeda dengan metode LSD yang memulai pengurutan dari digit paling kanan, MSD mengutamakan digit paling kiri sehingga sangat cocok digunakan pada data yang memiliki panjang digit tetap maupun data berbentuk string. Contoh data yang sesuai untuk metode ini antara lain NIM mahasiswa, nomor KTP, kode pos, nomor telepon, dan kode produk. Dalam proyek ini, MSD digunakan untuk mengurutkan data NIM mahasiswa.

Prinsip kerja MSD dilakukan dengan mengelompokkan seluruh data berdasarkan digit paling kiri. Apabila terdapat kelompok data yang memiliki digit awal sama, maka proses pengurutan dilanjutkan ke digit berikutnya secara rekursif hingga ditemukan perbedaan nilai. Setelah seluruh proses selesai, data akan tersusun sesuai urutan yang diinginkan. Karakteristik utama Radix Sort MSD adalah tidak menggunakan perbandingan langsung antar elemen, memanfaatkan konsep bucket atau kelompok digit, serta menggunakan pendekatan rekursif dalam proses pengurutan. Metode ini sangat efektif digunakan untuk data numerik maupun string yang memiliki struktur digit yang seragam.

2.3 Ternary Search

Ternary Search merupakan algoritma pencarian yang digunakan untuk menemukan suatu elemen pada kumpulan data yang telah terurut (*sorted data*). Algoritma ini merupakan pengembangan dari Binary Search dengan cara membagi ruang pencarian menjadi tiga bagian pada setiap iterasi menggunakan dua titik tengah, yaitu *mid1* dan *mid2*. Cara kerja Ternary Search dimulai dengan menentukan dua titik pembagi yang membagi data menjadi tiga bagian. Nilai yang dicari kemudian dibandingkan dengan elemen yang berada pada posisi *mid1* dan *mid2*. Jika nilai ditemukan pada salah satu titik tersebut, maka proses pencarian selesai. Namun jika tidak ditemukan, algoritma akan mempersempit ruang pencarian ke salah satu bagian yang masih memungkinkan mengandung data yang dicari. Proses ini dilakukan berulang hingga data ditemukan atau ruang pencarian habis. Agar dapat digunakan dengan baik, Ternary Search memiliki syarat bahwa data harus berada dalam kondisi terurut dan disimpan pada struktur data yang mendukung akses indeks secara langsung, seperti array atau vector. Oleh karena itu, pada penelitian ini data NIM mahasiswa terlebih dahulu diurutkan menggunakan algoritma Radix Sort sebelum dilakukan proses pencarian menggunakan Ternary Search. Penggunaan Ternary Search setelah proses pengurutan bertujuan untuk mempercepat pencarian data dibandingkan metode pencarian sederhana. Dengan membagi ruang pencarian menjadi tiga bagian pada setiap iterasi, jumlah langkah yang diperlukan untuk menemukan suatu data dapat dikurangi secara signifikan, terutama pada kumpulan data yang berukuran besar.

BAB III PSEUDOCODE ALGORITMA

3.1 Pseudocode Radix Sort LSD (Least Significant Digit)

Radix Sort LSD (*Least Significant Digit*) adalah algoritma pengurutan yang memproses data mulai dari digit paling kanan (digit satuan) menuju digit yang lebih kiri (puluhan, ratusan, dan seterusnya). Pada program yang dibuat, proses pengurutan menggunakan **Counting Sort** sebagai algoritma bantu pada setiap digit. Metode ini dipilih karena stabil (*stable sorting*) sehingga urutan data dengan digit yang sama tetap dipertahankan. Pada implementasi program, algoritma akan mencari nilai terbesar terlebih dahulu untuk menentukan jumlah digit maksimum yang harus diproses. Setelah itu, pengurutan dilakukan berulang kali mulai dari digit satuan hingga digit paling kiri.

Pseudocode Counting Sort

```
20
21  ✓ //Pengurutan Penghitungan untuk Pengurutan Radix LSD
22  ruang kosong menghitungSorot(vektor<panjang panjang>& arr, panjang panjangexp)
23  {
24      int n = arr.ukuran();
25
26      vektor<panjang panjang>keluaran(N);
27      int menghitung[10] = {0};
28
29      //Hitung frekuensi digit
30      untuk(int i = 0; i < n; i++)
31      {
32          hitung[(arr[i] / exp) % 10]++;
33          operasi++;
34      }
35
36      //Jumlah Awalan
37      untuk(int i = 1; saya < 10; i++)
38      {
39          count[i] += count[i - 1];
40          operasi++;
41      }
42  }
```


Pseudocode Radix Sort LSD

```
58     }
59     }
60
61     //LSD Radix Sort
62     ruang kosong radixSortLSD(vektor<panjang panjang>& arr)
63     {
64         panjang panjangNilai maksimum = *elemen_maksimum(arr.mulai(), aransemen.akhir());
65
66         intlulus =1;
67
68         untuk(panjang panjangexp =1; maxVal / exp >0; exp *=10)
69         {
70             menghitungSorot(arr, exp);
71
72             //Visualisasi hanya jika data <= 20
73             jika(arr.ukuran() <=20)
74             {
75                 cout <<"NLulus"<< lewati
76                     <<"(Angka"<< exp <<")\N";
77
78                 tampilkan(arr);
79             }
80         }
81     }
```

3.2 Pseudocode Radix Sort MSD (Most Significant Digit)

Radix Sort MSD (*Most Significant Digit*) merupakan algoritma pengurutan yang memulai proses dari digit paling kiri atau digit yang paling signifikan. Berbeda dengan LSD, MSD menggunakan pendekatan rekursif untuk mengurutkan setiap kelompok data yang memiliki digit awal yang sama. Pada implementasi program, data akan dimasukkan ke dalam beberapa bucket berdasarkan digit yang sedang diproses. Setelah itu setiap bucket yang berisi lebih dari satu elemen akan diurutkan kembali pada digit berikutnya hingga seluruh data berada dalam urutan yang benar.

Pseudocode MSD Radix Sort

```
138     }
139   ✓
140   #termasuk <iostream>
141   #termasuk <vektor>
142   #termasuk <rangkaian>
143
144   menggunakan ruang nama standar;
145
146   ruang kosong msdRadixSort(vektor<string>& arr, intkiri, intKanan, intangka)
147   {
148       jika(kiri >= kanan)
149           kembali;
150
151       intpanjang maksimum = arr[0].panjang();
152
153       jika(digit >= panjang maksimum)
154           kembali;
155
156       vektor<vektor<string>>keranjang(10);
157
158       untuk(inti = kiri; i <= kanan; i++)
159       {
160           intcurrentDigit = arr[i][digit] - '0';
161       }
```

Pseudocode Utama

```
196     {
197         msdRadixSort(arr,
198             R.Pertama,
199   ✓           R.Kedua,
200             angka +1);
201     }
202
```

3.3 Pseudocode Ternary Search

Ternary Search adalah algoritma pencarian yang digunakan pada data yang telah terurut. Berbeda dengan Binary Search yang membagi ruang pencarian menjadi dua bagian, Ternary Search membagi ruang pencarian menjadi tiga bagian menggunakan dua titik tengah, yaitu **mid1** dan **mid2**. Pada program ini, Ternary Search digunakan untuk mencari NIM mahasiswa setelah data diurutkan menggunakan Radix Sort.

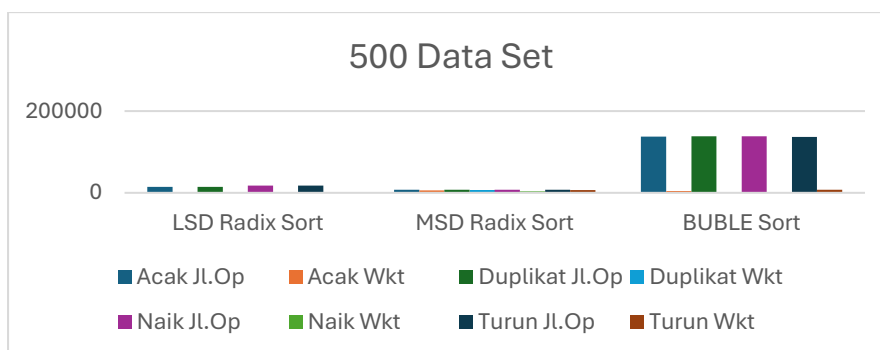
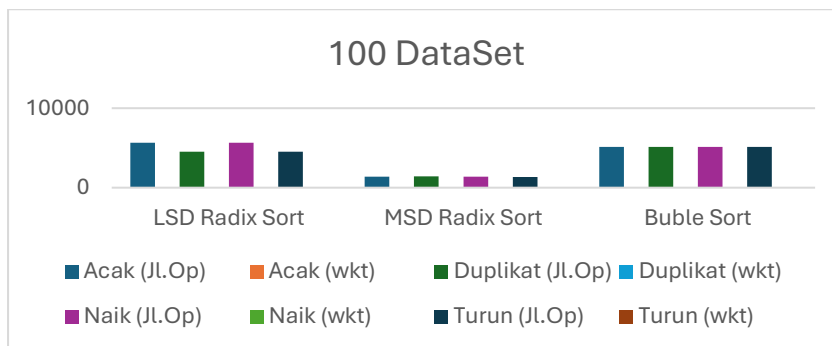
Pseudocode

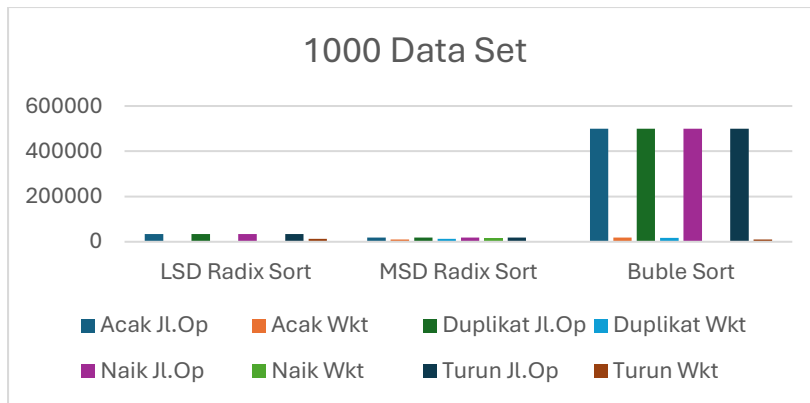
```
6
7 ✓ int Pencarian ternary(vektor<string>& arr, target string)
8 {
9     intkiri =0;
10    intkanan = arr.ukuran() -1;
11
12    ketika(kiri <= kanan)
13    {
14        inttengah1 = kiri + (kanan - kiri) /3;
15        inttengah2 = kanan - (kanan - kiri) /3;
16
17        cout <<"\N=====";
18        cout <<"\NKiri ="<< kiri;
19        cout <<"\NTengah1 ="<< tengah1 << "("<< arr[mid1] <<")";
20        cout <<"\NTengah2 ="<< tengah2 << "("<< arr[mid2] <<")";
21        cout <<"\Nkanan ="<< kanan << endl;
22
23        jika(arr[mid1] == target)
24            kembalitengah1;
25
26        jika(arr[mid2] == target)
27            kembalitengah2;
28
29
30
```

BAB IV DESAIN DAN IMPLEMENTASI PROGRAM

4.1 Arsitektur Program

Program yang dikembangkan merupakan aplikasi berbasis C++ yang digunakan untuk melakukan pengurutan (*sorting*) dan pencarian (*searching*) data NIM mahasiswa. Program mengintegrasikan beberapa modul yang saling terhubung, yaitu modul input data, modul sorting, modul searching, modul benchmark, dan modul ekspor data. Alur kerja program dimulai dari proses pemasukan data. Pengguna dapat memasukkan data secara manual, menghasilkan data acak, atau membaca data dari file teks. Selanjutnya data disimpan ke dalam memori menggunakan struktur data *vector*. Setelah data tersedia, pengguna dapat memilih algoritma pengurutan yang akan digunakan, yaitu Radix Sort LSD, Radix Sort MSD, atau Bubble Sort sebagai algoritma pembandingan. Setelah data berhasil diurutkan, pengguna dapat melakukan pencarian data menggunakan Ternary Search. Hasil pengolahan data kemudian dapat diekspor ke file CSV untuk keperluan dokumentasi dan analisis lebih lanjut.





4.2 Struktur Data yang Digunakan

Program menggunakan struktur data utama berupa `vector<string>`. Dengan code `vector<string> datasetUtama;` Struktur data ini digunakan untuk menyimpan seluruh data NIM mahasiswa yang akan diproses oleh program.

Penggunaan *vector* dipilih karena memiliki beberapa keunggulan, yaitu:

1. Ukuran penyimpanan dapat bertambah secara dinamis.
2. Mendukung akses data menggunakan indeks.
3. Mempermudah proses pengurutan dan pencarian.
4. Efisien untuk menyimpan data dalam jumlah besar.

Selain itu, algoritma Radix Sort menggunakan struktur data tambahan berupa **bucket** untuk mengelompokkan data berdasarkan digit tertentu.

Contoh implementasi pada MSD:

```
102         vector<vector<string>> bucket(256);
```

Sedangkan pada LSD digunakan array penghitung (*count array*) sebagai bagian dari Counting Sort.

```
int count[256] = {0};
```

4.3.1 Modul Input Data

Modul input data digunakan untuk memasukkan data ke dalam sistem. Program menyediakan tiga metode input yaitu:

- Input manual

- Generate data acak
- Membaca data dari file teks (.txt)

Modul ini bertujuan untuk memberikan fleksibilitas kepada pengguna dalam menyiapkan data pengujian.

4.3.2 Modul Radix Sort LSD

Modul Radix Sort LSD mengurutkan data mulai dari digit paling kanan menuju digit paling kiri. Algoritma ini menggunakan Counting Sort pada setiap digit sehingga proses pengurutan dapat dilakukan tanpa perbandingan langsung antar data.

Fitur yang tersedia:

- Menampilkan trace setiap pass.
- Menghitung jumlah operasi.
- Mengukur waktu eksekusi.

4.3.3 Modul Radix Sort MSD

Modul Radix Sort MSD mengurutkan data mulai dari digit paling kiri menggunakan pendekatan rekursif. Data dikelompokkan ke dalam bucket berdasarkan digit yang sedang diproses kemudian diurutkan kembali hingga seluruh digit selesai diproses.

Fitur yang tersedia:

- Pengelompokan data menggunakan bucket.
- Pengurutan rekursif.
- Menampilkan proses pengurutan setiap digit.

4.3.4 Modul Bubble Sort

Bubble Sort digunakan sebagai algoritma pembandingan dalam eksperimen. Algoritma ini bekerja dengan membandingkan dua elemen yang berdekatan kemudian menukarnya apabila urutannya salah.

Meskipun sederhana, Bubble Sort memiliki kompleksitas waktu $O(n^2)$ sehingga digunakan sebagai pembandingan untuk melihat keunggulan Radix Sort.

4.3.5 Modul Ternary Search

Modul Ternary Search digunakan untuk mencari data setelah data berada dalam keadaan terurut. Algoritma ini membagi ruang pencarian menjadi tiga bagian menggunakan dua titik tengah (*mid1* dan *mid2*) sehingga proses pencarian menjadi lebih efisien.

4.3.6 Modul Export CSV

Modul ini digunakan untuk menyimpan hasil pengolahan data ke dalam file CSV sehingga dapat digunakan untuk dokumentasi maupun analisis lanjutan.

4.4 Tampilan Program Saat Dieksekusi

Tampilan Menu Utama

```
=====
SISTEM GABUNGAN SORTING & SEARCHING KELOMPOK
=====

1. Input Data Manual (String/NIM)
2. Generate Data Angka/NIM Acak
3. Baca Data dari File (.txt)
4. Tampilkan Data Saat Ini
5. Uji Eksperimen Radix Sort LSD
6. Uji Eksperimen Radix Sort MSD
7. Jalankan Bubble Sort (Pembanding)
8. Cari Data (Ternary Search)
9. Export Hasil Data ke CSV
0. Keluar
```

Tampilan Hasil Pengurutan LSD

Contoh hasil pengurutan: Hasil pengurutan menggunakan Radix Sort LSD.

[Hasil Akhir Terurut LSD]

231401001
231401002
231401003
231401007
231401010
231401015
231401020

Jumlah Operasi : 5650

Waktu Eksekusi : 0 microseconds

Tampilan Hasil Pengurutan MSD

Contoh hasil pengurutan: Hasil pengurutan menggunakan Radix Sort MSD.

[Hasil Akhir Terurut MSD]

231401001
231401002
231401003
231401007
231401010
231401015
231401020

Jumlah Operasi : 1382

Waktu Eksekusi : 0 microseconds

Tampilan Hasil Pencarian Ternary Search

Contoh hasil pencarian: Hasil pencarian menggunakan Ternary Search.

Masukkan NIM yang dicari :

231401015

NIM ditemukan pada indeks 5

Tampilan Hasil Export CSV

Hasil ekspor data ke file CSV.

No	Data
1	231401001
2	231401002
3	231401003
...	

4.5 Hasil Implementasi dan Pengujian

Berdasarkan hasil benchmark yang diperoleh, Radix Sort MSD menunjukkan jumlah operasi yang lebih kecil dibandingkan LSD Radix Sort dan Bubble Sort pada sebagian besar kondisi data. Pada pengujian data acak, MSD Radix Sort menghasilkan sekitar **1.382 operasi**, sedangkan LSD Radix Sort menghasilkan **5.650 operasi** dan Bubble Sort menghasilkan **5.117 operasi**. Hasil ini menunjukkan bahwa pendekatan pengelompokan berdasarkan digit paling signifikan mampu mengurangi jumlah operasi yang dilakukan selama proses pengurutan.

Pada pengujian dengan ukuran data yang lebih besar, perbedaan performa semakin terlihat. Bubble Sort menghasilkan jumlah operasi yang jauh lebih tinggi dibandingkan kedua algoritma Radix Sort. Hal ini sesuai dengan teori bahwa Bubble Sort memiliki kompleksitas waktu $O(n^2)$, sedangkan Radix Sort bekerja berdasarkan jumlah digit dan jumlah data sehingga lebih efisien untuk pengolahan data NIM mahasiswa dalam jumlah besar.

BAB V STUDI KASUS

5.1 Deskripsi Kasus Nyata

Dalam lingkungan perguruan tinggi, data mahasiswa merupakan salah satu jenis data yang paling sering dikelola. Setiap mahasiswa memiliki identitas unik berupa **Nomor Induk Mahasiswa (NIM)** yang digunakan dalam berbagai kegiatan akademik, seperti pengisian Kartu Rencana Studi (KRS), pengolahan nilai, pendaftaran ujian, administrasi akademik, hingga pencarian data mahasiswa pada sistem informasi kampus.

Seiring bertambahnya jumlah mahasiswa setiap tahun, jumlah data NIM yang tersimpan dalam basis data juga semakin besar. Kondisi ini menuntut adanya metode pengolahan data yang cepat dan efisien, terutama dalam proses pengurutan (*sorting*) dan pencarian (*searching*) data. Jika proses pengurutan dan pencarian dilakukan menggunakan algoritma yang kurang efisien, maka waktu pemrosesan akan meningkat dan dapat mempengaruhi kinerja sistem secara keseluruhan.

Berdasarkan permasalahan tersebut, penelitian ini mengimplementasikan algoritma **Radix Sort LSD (Least Significant Digit)** dan **Radix Sort MSD (Most Significant Digit)** untuk melakukan pengurutan data NIM mahasiswa. Selain itu, digunakan algoritma **Ternary Search** untuk melakukan pencarian data setelah proses pengurutan selesai dilakukan. Sebagai pembanding, digunakan algoritma **Bubble Sort** untuk melihat perbedaan performa antara algoritma sorting modern dan algoritma sorting sederhana.

Melalui studi kasus ini, diharapkan dapat diketahui algoritma yang paling efektif dalam mengolah data NIM mahasiswa berdasarkan jumlah operasi dan waktu eksekusi yang dihasilkan.

5.2 Dataset yang Digunakan

Dataset yang digunakan dalam penelitian ini berupa data NIM mahasiswa yang direpresentasikan dalam bentuk angka dan disimpan sebagai string pada program. Data diperoleh melalui proses pembangkitan data (*generate random data*) yang dibuat oleh program serta beberapa variasi kondisi data untuk kebutuhan pengujian performa algoritma.

Pengujian dilakukan menggunakan empat kondisi dataset, yaitu:

1. Data Acak (Random)

Data NIM disusun secara acak tanpa pola tertentu.

Contoh:

```
231401015
231401003
231401020
231401001
231401010
```

Kondisi ini digunakan untuk mensimulasikan data mahasiswa yang belum pernah diurutkan sebelumnya.

2. Data Terurut Naik (Ascending)

Data NIM sudah berada dalam kondisi terurut dari nilai terkecil ke terbesar.

Contoh:

```
231401001
231401002
231401003
231401004
231401005
```

Kondisi ini digunakan untuk mengetahui performa algoritma ketika menerima data yang telah terurut.

3. Data Terurut Turun (Descending)

Data NIM disusun dari nilai terbesar ke nilai terkecil.

Contoh:

```
231401005
231401004
231401003
231401002
231401001
```

Kondisi ini sering dianggap sebagai kondisi terburuk bagi beberapa algoritma sorting.

4. Data Duplikat (Duplicate)

Dataset mengandung beberapa nilai yang sama.

Contoh:

```
231401001
231401001
231401003
231401003
231401005
```

Kondisi ini digunakan untuk menguji kemampuan algoritma dalam menangani data yang memiliki nilai berulang.

Ukuran dataset yang digunakan dalam eksperimen adalah:

Jumlah Data

100

500

1000

5000

10000

Berbagai ukuran data tersebut digunakan untuk melihat pengaruh jumlah data terhadap performa masing-masing algoritma.

5.3 Pembahasan Hasil Pengujian

Berdasarkan hasil benchmark yang diperoleh, algoritma **Radix Sort MSD** menunjukkan performa yang sangat baik pada sebagian besar kondisi data. Jumlah operasi yang dilakukan MSD lebih sedikit dibandingkan LSD maupun Bubble Sort, terutama ketika ukuran data semakin besar. Hal ini terjadi karena MSD memproses digit paling signifikan terlebih dahulu sehingga proses pengelompokan data menjadi lebih efektif.

Algoritma **Radix Sort LSD** juga menghasilkan performa yang baik dan relatif stabil pada berbagai kondisi dataset. LSD bekerja dengan memproses digit dari kanan ke kiri menggunakan Counting Sort sehingga mampu mengurutkan data dalam waktu yang singkat. Meskipun jumlah operasinya lebih besar dibandingkan MSD, performanya tetap jauh lebih baik dibandingkan Bubble Sort.

Sementara itu, **Bubble Sort** menghasilkan jumlah operasi yang paling besar. Pada ukuran data kecil perbedaannya belum terlalu terlihat, tetapi ketika jumlah data meningkat hingga ribuan data, jumlah operasi yang dilakukan Bubble Sort meningkat secara signifikan. Hal ini sesuai dengan teori bahwa Bubble Sort memiliki kompleksitas waktu sebesar $O(n^2)$.

Untuk proses pencarian data, algoritma **Ternary Search** mampu menemukan data dengan cepat setelah data berada dalam kondisi terurut. Dengan membagi ruang pencarian menjadi tiga bagian pada setiap iterasi, jumlah langkah pencarian menjadi lebih sedikit dibandingkan pencarian linear (*Linear Search*).

Berdasarkan keseluruhan hasil eksperimen, dapat disimpulkan bahwa kombinasi **Radix Sort (LSD dan MSD)** serta **Ternary Search** sangat cocok digunakan untuk pengolahan data NIM mahasiswa dalam jumlah besar karena mampu memberikan performa yang lebih cepat dan efisien dibandingkan metode sorting sederhana seperti Bubble Sort.

BAB VI ANALISIS HASIL EKSPERIMEN

6.1 Hasil Pengujian dan Perbandingan Performa

Pengujian dilakukan untuk membandingkan performa tiga algoritma pengurutan, yaitu **Radix Sort LSD**, **Radix Sort MSD**, dan **Bubble Sort**. Pengujian menggunakan empat kondisi dataset, yaitu data acak (*random*), data terurut naik (*ascending*), data terurut turun (*descending*), dan data duplikat (*duplicate*). Ukuran data yang digunakan adalah 100 data NIM mahasiswa.

Parameter yang digunakan dalam analisis adalah **jumlah operasi** yang dilakukan selama proses pengurutan. Semakin sedikit jumlah operasi yang dilakukan, maka semakin efisien algoritma tersebut.

Perbandingan Jumlah Operasi Algoritma Sorting

Kondisi Data LSD Radix Sort MSD Radix Sort Bubble Sort

Acak	5.650	1.382	5.117
Duplikat	5.650	1.383	5.086
Terurut Naik	5.650	1.383	4.950
Terurut Turun	5.650	1.383	9.900

Dari data tersebut terlihat bahwa jumlah operasi pada Radix Sort LSD relatif tetap pada setiap kondisi data. Hal ini menunjukkan bahwa performa LSD tidak dipengaruhi oleh susunan awal data karena algoritma bekerja berdasarkan digit, bukan berdasarkan perbandingan antar elemen.

Sebaliknya, MSD Radix Sort menghasilkan jumlah operasi yang jauh lebih kecil dibandingkan LSD maupun Bubble Sort. Jumlah operasi MSD berada pada kisaran 1.382–1.383 operasi untuk seluruh kondisi data yang diuji.

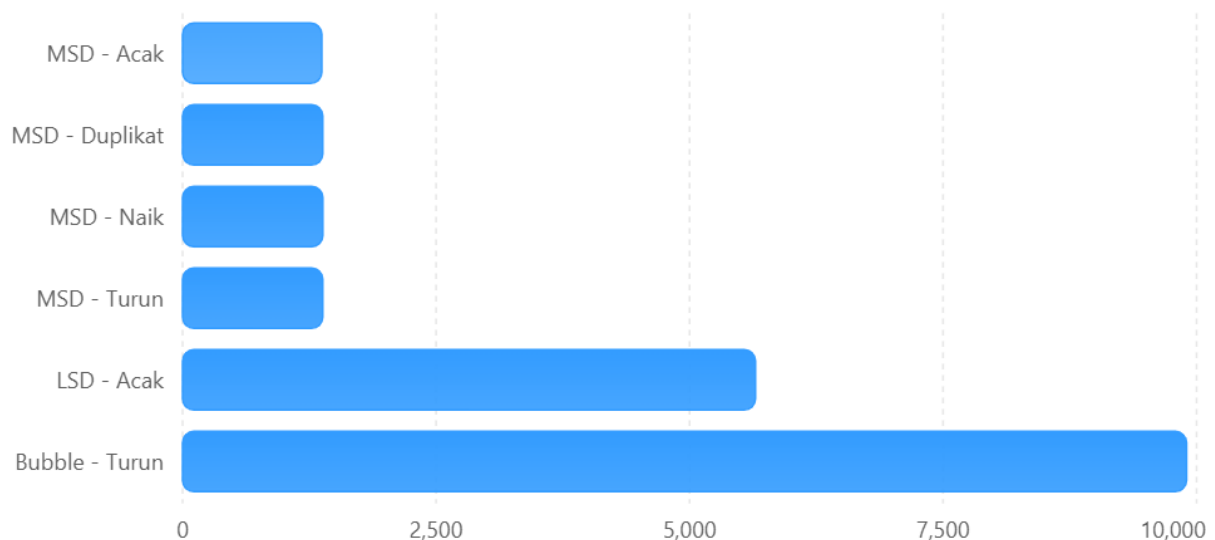
Bubble Sort menunjukkan performa yang paling rendah. Pada kondisi data terurut turun (*descending*), jumlah operasi meningkat hingga 9.900 operasi, yang merupakan nilai tertinggi dibandingkan algoritma lainnya.

6.2 Grafik Perbandingan Performa Algoritma

Grafik Jumlah Operasi pada Berbagai Kondisi Data

Perbandingan Jumlah Operasi Algoritma Sorting

Jumlah operasi pada data berukuran 100 dengan berbagai kondisi data.



Berdasarkan grafik di atas, MSD Radix Sort memiliki jumlah operasi paling rendah sehingga menjadi algoritma yang paling efisien dalam pengujian ini. Bubble Sort pada kondisi data terurut turun menghasilkan jumlah operasi tertinggi sehingga menjadi algoritma yang paling lambat.

6.3 Analisis Temuan

1. Radix Sort MSD Merupakan Algoritma Paling Efisien

Hasil eksperimen menunjukkan bahwa MSD Radix Sort menghasilkan jumlah operasi paling sedikit dibandingkan algoritma lainnya. Hal ini disebabkan karena MSD memproses digit paling signifikan terlebih dahulu sehingga data dapat dikelompokkan lebih cepat ke dalam bucket yang sesuai. Proses rekursif yang digunakan juga membantu mengurangi jumlah operasi yang tidak diperlukan.

2. Radix Sort LSD Memiliki Performa Stabil

Radix Sort LSD menunjukkan jumlah operasi yang sama pada seluruh kondisi data, yaitu sekitar 5.650 operasi. Kondisi ini menunjukkan bahwa performa LSD tidak dipengaruhi oleh

urutan awal data. Baik data acak, terurut, terbalik, maupun duplikat menghasilkan jumlah operasi yang hampir identik.

3. Bubble Sort Sangat Dipengaruhi Kondisi Data

Bubble Sort menghasilkan performa yang berbeda pada setiap kondisi data. Ketika data sudah terurut naik, jumlah operasi relatif lebih kecil. Namun pada kondisi data terurut turun, jumlah operasi meningkat drastis hingga mencapai 9.900 operasi. Hal ini sesuai dengan karakteristik Bubble Sort yang memiliki kompleksitas waktu $O(n^2)$.

4. Pengaruh Ukuran Data terhadap Performa

Semakin besar jumlah data yang diproses, perbedaan performa antar algoritma akan semakin terlihat. Radix Sort LSD dan MSD tetap mampu mempertahankan performa yang baik karena bekerja berdasarkan digit data. Sebaliknya, Bubble Sort akan mengalami peningkatan jumlah operasi secara signifikan seiring bertambahnya ukuran data.

5. Efektivitas Ternary Search

Setelah data berhasil diurutkan menggunakan Radix Sort, proses pencarian menggunakan Ternary Search dapat dilakukan dengan cepat. Algoritma ini membagi ruang pencarian menjadi tiga bagian sehingga jumlah langkah pencarian menjadi lebih sedikit dibandingkan pencarian linear.

6.4 Kesimpulan Hasil Eksperimen

Berdasarkan hasil pengujian dan analisis yang dilakukan, dapat disimpulkan bahwa **Radix Sort MSD merupakan algoritma yang paling efisien** dalam pengurutan data NIM mahasiswa karena menghasilkan jumlah operasi paling sedikit. **Radix Sort LSD** juga menunjukkan performa yang baik dan stabil pada berbagai kondisi data. Sementara itu, **Bubble Sort** memiliki performa paling rendah karena jumlah operasi meningkat secara signifikan ketika ukuran data bertambah atau ketika data berada dalam kondisi terurut turun.

Hasil penelitian menunjukkan bahwa penggunaan algoritma **Radix Sort (LSD dan MSD)** yang dikombinasikan dengan **Ternary Search** sangat cocok diterapkan pada sistem pengolahan data mahasiswa karena mampu memberikan proses pengurutan dan pencarian yang lebih cepat, efisien, dan stabil dibandingkan algoritma sorting sederhana seperti Bubble Sort.

BAB VII KESIMPULAN

7.1 Kesimpulan

Berdasarkan hasil implementasi, pengujian, dan analisis yang telah dilakukan terhadap algoritma **Radix Sort LSD, Radix Sort MSD, Bubble Sort, dan Ternary Search**, diperoleh beberapa kesimpulan sebagai berikut:

1. **Algoritma Radix Sort berhasil digunakan untuk mengurutkan data NIM mahasiswa dengan baik dan akurat.** Baik metode LSD (*Least Significant Digit*) maupun MSD (*Most Significant Digit*) mampu menghasilkan data yang terurut secara benar sesuai urutan menaik (*ascending*).
2. **Radix Sort MSD menunjukkan performa terbaik dibandingkan algoritma lainnya.** Berdasarkan hasil eksperimen pada dataset berjumlah 100 data, MSD Radix Sort menghasilkan jumlah operasi sekitar **1.382–1.383 operasi**, lebih sedikit dibandingkan LSD Radix Sort yang memerlukan sekitar **5.650 operasi** dan Bubble Sort yang memerlukan hingga **9.900 operasi** pada kondisi data tertentu.
3. **Radix Sort LSD memiliki performa yang stabil pada berbagai kondisi data.** Jumlah operasi yang dihasilkan relatif tetap pada data acak, data duplikat, data terurut naik, maupun data terurut turun. Hal ini menunjukkan bahwa performa LSD tidak terlalu dipengaruhi oleh susunan awal data.
4. **Bubble Sort memiliki performa paling rendah untuk pengolahan data dalam jumlah besar.** Jumlah operasi yang dilakukan meningkat secara signifikan terutama pada kondisi data terurut turun. Hal ini sesuai dengan karakteristik Bubble Sort yang memiliki kompleksitas waktu $O(n^2)$.
5. **Ternary Search mampu melakukan pencarian data secara efisien setelah data diurutkan terlebih dahulu.** Dengan membagi ruang pencarian menjadi tiga bagian pada setiap iterasi, proses pencarian menjadi lebih cepat dibandingkan metode pencarian linear.
6. Berdasarkan keseluruhan hasil pengujian, kombinasi **Radix Sort (LSD dan MSD)** dengan **Ternary Search** terbukti efektif untuk pengolahan data NIM mahasiswa karena mampu memberikan proses pengurutan dan pencarian yang cepat, efisien, serta menghasilkan jumlah operasi yang lebih kecil dibandingkan algoritma sorting konvensional.

7.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran untuk pengembangan lebih lanjut, yaitu:

1. Pengujian dapat dilakukan menggunakan jumlah data yang lebih besar, seperti 10.000 hingga 100.000 data, sehingga performa algoritma dapat dianalisis secara lebih mendalam.
2. Dataset yang digunakan dapat diperluas dengan menggunakan data nyata dari institusi pendidikan atau dataset publik sehingga hasil penelitian lebih merepresentasikan kondisi dunia nyata.
3. Penelitian selanjutnya dapat menambahkan algoritma sorting lain, seperti Quick Sort, Merge Sort, atau Heap Sort sebagai pembanding untuk memperoleh analisis performa yang lebih komprehensif.
4. Program dapat dikembangkan menjadi aplikasi berbasis antarmuka grafis (*Graphical User Interface/GUI*) atau berbasis web agar lebih mudah digunakan oleh pengguna.
5. Fitur visualisasi proses sorting dan searching dapat diperbaiki sehingga pengguna dapat melihat setiap tahapan algoritma secara lebih interaktif dan mudah dipahami.

Rekomendasi Penggunaan Algoritma

Berdasarkan hasil eksperimen yang diperoleh, **Radix Sort MSD direkomendasikan sebagai algoritma utama untuk pengurutan data NIM mahasiswa dalam jumlah besar** karena menghasilkan jumlah operasi paling sedikit dan menunjukkan performa terbaik. **Radix Sort LSD** dapat digunakan sebagai alternatif karena memiliki performa yang stabil dan implementasi yang relatif sederhana. Sementara itu, **Bubble Sort lebih cocok digunakan untuk tujuan pembelajaran atau pengujian dasar**, bukan untuk pengolahan data berskala besar karena efisiensinya lebih rendah dibandingkan Radix Sort.

Untuk proses pencarian data yang sudah terurut, **Ternary Search direkomendasikan sebagai metode pencarian** karena mampu menemukan data dengan cepat dan efisien melalui pembagian ruang pencarian menjadi tiga bagian pada setiap iterasi.

REFERENSI

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). Cambridge, MA: The MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in C++ (2nd ed.). Hoboken, NJ: John Wiley & Sons.

Princeton University. (n.d.). Algorithms, 4th Edition Documentation: LSD Radix Sort and MSD Radix Sort. Diakses pada Juni 2026 dari <https://algs4.cs.princeton.edu>

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Boston, MA: Addison-Wesley Professional.

Weiss, M. A. (2014). Data Structures and Algorithm Analysis in C++ (4th ed.). Boston, MA: Pearson Education.

Knuth, D. E. (1998). The Art of Computer Programming, Volume 3: Sorting and Searching (2nd ed.). Reading, MA: Addison-Wesley Professional.